Insertion Sort

Sorting the whitelist: the first sorting method, its cost, and why it does not scale

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026 · Sedgewick & Wayne, *Introduction to Programming in Java* 2e, §4.2

By the end of this session you can

1. state the sorting problem and name applications
2. explain insertion sort with its invariant: `a[0, i)` is sorted
3. trace insertion sort by hand and count its exchanges
4. implement `Insertion.sort()` with an early `break`
5. derive its cost: $\sim N$ (sorted), $\sim N^2/4$ (random), $\sim N^2/2$ (reversed)
6. confirm quadratic growth with a doubling test
7. explain why a quadratic algorithm does not scale

Recap and bridge: binary search needs a sorted array

Binary search

$\sim$lg N compares per search ... but only on a **sorted** array. `TestBS` called `Arrays.sort()` first.

Last checkout question

How do you sort 15 name cards by hand?

Pick up one card at a time and slide it into its place among the cards already in your hand.

That is **insertion sort** – today's algorithm.

Sorting: rearrange N items into ascending order

Applications

- binary search
- statistics, databases
- data compression, bioinformatics
- computer graphics, scientific computing
- GenAI: ranking the likeliest next words
- ... too numerous to list

0	wendy		0	alice
1	alice		1	bob
2	dave		2	carlos
3	walter		3	carol
4	carlos		4	craig
5	carol		5	dave
6	erin	$\rightarrow$	6	erin
7	oscar		7	eve
8	peggy		8	frank
9	trudy		9	oscar
10	eve		10	peggy
11	trent		11	trent
12	bob		12	trudy
13	craig		13	victor
14	frank		14	walter
15	victor		15	wendy

Sedgewick: a substantial share of all computing time goes into sorting

Pop quiz: guess the time

How long does a laptop take to sort
one million random 32-bit integers?

(a) a few milliseconds
(b) about a tenth of a second
(c) a few minutes
(d) several hours

Measured on a desktop PC

`Arrays.sort(int[])`: **0.05 s** – (b)
insertion sort: **96 s** – (c), 2,000 times slower

Better than both

Integers have digits: **radix sort** (one byte at a time, 4 passes) runs in linear time – no compares at all.

Desktop PC: Intel Core i7-14700K (2023), 20 cores – each sort uses one of them

Insertion sort: the idea

Like sorting a hand of cards

- Move down through the array: i = 1, 2, . . . , N−1
- a[i] moves left, exchanged with each **larger** neighbor
- It stops at the first neighbor that is not larger
- Everything left of i is in order: `a[0, i)` is sorted
- Everything right of i is untouched

Not bubble sort

"Like bubble sort, but not bubble sort. We don't teach bubble sort any more because this is simpler and faster."

– Sedgewick

Insertion sort vs. bubble sort (optional)

	insertion sort	bubble sort
sorted part	left: `a[0, i)`	right: the largest move to the end
inner loop stops	when the entry is in place	at the end of the unsorted part
sorted input	$\sim N$ compares	$\sim N^2/2$ compares
exchanges	one per out-of-order pair	the same

Bubble sort wastes compares on pairs that are already in order.

Trace: the 16 names, first insertions

i	j	inserted	a[0, i] after the insertion
1	0	alice	alice wendy
2	1	dave	alice dave wendy
3	2	walter	alice dave walter wendy
4	1	carlos	alice carlos dave walter wendy
5	2	carol	alice carlos carol dave walter wendy
			...
12	1	bob	moves 11 places
15	13	victor	moves 2 places

j is where the inserted name stopped: the cost depends on the data.

All 15 insertions: companion `birkenkrahe.github.io/demos/insertion-sort.html`

Insertion: sort the 16 names

→ **Switch to editor · code-along 1**

1. Write **`sort(String[] a)`**: outer loop from `i = 1`, inner loop from `j = i` down
2. Compare **`a[j-1]`** with **`a[j]`**: exchange if larger, otherwise **`break`**
3. Write **`exch(a, i, j)`**
4. Test client: read all strings, sort, print
5. Run it on **`names16.txt`**

files: `Insertion.java` · `names16.txt` · classpath: `.:stdlib.jar`

The code (Program 4.2.4)

```
public static void sort(String[] a)
{
    int N = a.length;
    for (int i = 1; i < N; i++)
        for (int j = i; j > 0; j--)
            if (a[j-1].compareTo(a[j]) > 0)
                exch(a, j-1, j);
            else break;
}
private static void exch(String[] a,
                         int i, int j)
{ String t = a[i]; a[i] = a[j]; a[j] = t; }
```

Why start at `i = 1`?
→ One entry is always sorted.

What does the `break` do?
→ Stops as soon as the entry is in place. Without it: still correct, but $\sim N^2/2$ compares even on sorted input.

The book sorts `Comparable[]` and tests `a[j].compareTo(a[j-1]) < 0` – the same comparison

Correctness: the same three pieces as binary search

piece	what it guarantees	in `Insertion.sort()`
invariant	nothing is lost on the way	at the start of each pass, `a[0, i)` is sorted
termination	an answer is always reached	`i` grows by one; `j` moves down to 0 at most
exit	the answer is right	at `i == N`, `a[0, N)` – the whole array – is sorted

The invariant holds at each check of the outer loop – not while an entry is being moved.

Analysis: the cost depends on the input

input	each insertion	compares
sorted	one compare, then `break`	$N-1 \sim N$
reversed	`a[i]` moves all the way: `i` exchanges	$1+2+\cdots+(N-1) \sim N^2/2$
random	`a[i]` moves about halfway	$\sim N^2/4$

Each exchange fixes one inversion

an **inversion** is a pair that is out of order – a reversed array has all $N(N-1)/2$ of them

Book pp. 544–545: the trace as an $N \times N$ square, half of the entries below the diagonal marked

Counting demo: `InsertionCount` (optional)

Sort the same 4,000 random strings three ways and count:

```
sorted   compares     3,999  exchanges         0
random   compares 4,022,953  exchanges 4,018,958
reversed compares 7,998,000  exchanges 7,998,000
                  N^2/4 = 4,000,000   N^2/2 = 8,000,000
```

Sensitive to input (book, p. 548)

Linear on nearly sorted data, quadratic on random data. That is why library sorts (TimSort) use insertion sort for short runs.

TestInsertion: the doubling test

→ **Switch to editor · code-along 2**

1. Copy **TestBS** to **TestInsertion**: read the strings, time only **Insertion.sort(a)**
2. **Predict:** doubling N multiplies the time by about 4
3. Run N = 20,000, 40,000, 80,000, 160,000 with `Generator`
4. Compute the ratios

files: `TestInsertion.java` · `Insertion.java` · `Generator.java`

Results: ratio $\to$ 4, quadratic

Sedgewick

N	T(N) s	ratio
20,000	1	
40,000	4	4
80,000	35	9
160,000	225	6.4
320,000	1019	4.5
...		
1.28 million	14400	4

Desktop PC, 2026

N	T(N) s	ratio
20,000	0.22	
40,000	1.22	5.5
80,000	6.09	5.0
160,000	27.13	4.5
320,000	131.30	4.8

1.28 million strings: 4 hours. 10 million: about $4^3 = 64$ times as long – more than 10 days.
Insertion sort will not scale.

Desktop PC: Intel Core i7-14700K (2023), one core

A rule of thumb: Moore's law

Moore's law

The number of transistors on a chip doubles about every two years.

Sedgewick's implications:

- memory doubles every two years
- processor speed doubles every two years

Rule of thumb: touching every word of memory takes a few seconds.

computer	instr./s	words
PDP-9	10^4	10^4
VAX 11-780	10^6	10^6
CRAY 1	10^7	10^7
laptop	10^9	10^9

Week 5: a faster machine changes the constant a in $T(N) \sim aN^b$, never the exponent b.

Gordon Moore, Intel co-founder, 1929–2023

Scalability

An algorithm scales if its running time doubles when the problem size doubles.

New computer: 2× speed **and** 2× memory, so problems grow to 2N. New time = doubling ratio / 2:

order of growth	ratio	new time	scales?
N	2	the same	yes
N log N	$\gtrsim 2$	a bit more	yes
N^2 (insertion)	4	**twice as long**	no
N^3	8	four times as long	no

Since about 2005 single processors barely get faster; the growth comes as more cores. Our desktop PC has 20 cores – insertion sort uses one. You need algorithms that scale, and that can run in parallel.

Beyond strings: `Comparable` (optional)

The book's `Insertion` (p. 547) sorts any comparable type:

```
public static void sort(Comparable[] a)
...
    if (a[j].compareTo(a[j-1]) < 0) exchange(a, j-1, j);
```

- `String`, `Integer`, `Double` are `Comparable`
- Your own type: `implements Comparable<Counter>` plus a `compareTo()`
- `compareTo()` must define a **total order**: antisymmetric, transitive, total (p. 546)
- As in binary search: test the sign, never -1 or 1

Practice: trace insertion sort by hand

PRACTICE · CLOSED BOOK · 10 MIN

Sort these 8 words with insertion sort:

```
was had him and you his the but
```

1. After each value of i = 1 . . . 7, write down the array
2. Count the exchanges in each pass, then add them up
3. Check your trace with **`InsertionTrace < 8words.txt`**

Compare your total with $N^2/4 = 16$.

Book: the same trace on p. 544

Interactive companion

birkenkrahe.github.io/demos/insertion-sort.html

- The 16 names, one compare per click, with the code
- The trace builds row by row, as on the slide and in the book
- Sorted, random and reversed input: $\sim N$, $\sim N^2/4$, $\sim N^2/2$

Wrap-up: insertion sort

- Sorting: rearrange N items into ascending order
- Insertion sort: insert `a[i]` into the sorted `a[0, i)`; stop when it is in place
- Correct: invariant, termination, exit – as for binary search
- Cost depends on the input: $\sim N$, $\sim N^2/4$, $\sim N^2/2$
- Doubling ratio 4: quadratic – it does not scale

Next: an N log N sort – mergesort.

Checkout question

You have 1,000 exam papers to sort by name, and two friends to help. How do you split the work?